

3.1 An operating system contains 3 user processes each requiring 2 units of resource R. the minimum number of units of r such that no deadlocks will ever arise is

- (a) 3 (b) 5
(c) 4 (d) 6 [1997 : 2 M]

3.2 A computer has six tape drives, with n processes competing for them. Each process may need two drives. What is the maximum value of n for the system to be deadlock free?

- (a) 6 (b) 5
(c) 4 (d) 3 [1998 : 1 M]

3.3 Which of the following is NOT a valid deadlock prevention scheme?

- (a) Release all resources before requesting a new resource.
(b) Number the resources uniquely and never request a lower numbered resource than the last one requested.
(c) Never request a resource after releasing any resource
(d) Request and all required resources be allocated before execution. [2000 : 2 M]

3.4 In a certain operating system, deadlock prevention is attempted using the following scheme. Each process is assigned a unique timestamp, and is restarted with the same timestamp if killed. Let P_h be the process holding a resource R, P_r be a process requesting for the same resource R, and $T(P_h)$ and $T(P_r)$ be their timestamps respectively. The decision to wait or preempt one of the processes is based on the following algorithm.

```
if  $T(P_r) < T(P_h)$  then
  kill  $P_r$ 
else
  wait
```

Which one of the following is TRUE?

- (a) The scheme is deadlock-free, but not starvation-free
(b) The scheme is not deadlock-free, but starvation-free
(c) The scheme is neither deadlock-free nor starvation-free

(d) The scheme is both deadlock-free and starvation-free [2004 : 2 M]

3.5 Suppose n processes, $P_1, \dots, P_n$ share m identical resource units, which can be reserved and released one at a time. The maximum resource requirement of process P_i is s_i where $s_i > 0$. Which one of the following is a sufficient condition for ensuring that deadlock does not occur?

- (a) $\forall i, s_i < m$ (b) $\forall i, s_i < n$
(c) $\sum_{i=1}^n s_i < (m + n)$ (d) $\sum_{i=1}^n s_i < (m * n)$

[2005 : 2 M]

3.6 Consider the solution to the bounded buffer producer/consumer problem by using general semaphores S, F, and E. The semaphore S is the mutual exclusion semaphore initialized to 1. The semaphore F corresponds to the number of free slots in the buffer and is initialized to N. The semaphore E corresponds to the number of elements in the buffer and is initialized to 0.

Producer Process	Consumer Process
Produce an item;	Wait (E);
Wait (F);	Wait (S);
Wait (S);	Remove an item from the buffer;
Append the item to the buffer;	Signal (S);
Signal (S);	Signal (F);
Signal (E);	Consumer the item;

Which of the following interchange operations may result in a deadlock?

- I. Interchanging Wait (F) and Wait (S) in the Producer process.
II. Interchanging Signal (S) and Signal (F) in the Consumer process.
(a) I only (b) II only
(c) Neither I nor II (d) Both I and II [2006 : 2 M]

3.7 Consider the following snapshot of a system running n processes. Process i is holding x_i instances of a resource R, for $1 \leq i \leq n$. Currently, all instances of R are occupied. Further, for all i,

process i has placed a request for an additional y_i instances while holding the x_i instances it already has. There are exactly two processes p and q such that $y_p = y_q = 0$. Which one of the following can serve as a necessary condition to guarantee that the system is not approaching a deadlock?

- (a) $\min(x_p, x_q) < \max_{k \neq p, q} y_k$
- (b) $x_p + x_q \geq \max_{k \neq p, q} y_k$
- (c) $\min(x_p, x_q) < 1$
- (d) $\min(x_p, x_q) > 1$

[2006 : 2 M]

3.8 A single processor system has three resource types X, Y, and Z, which are shared by three processes. There are 5 units of each resource type. Consider the following scenario, where the column alloc denotes the number of units of each resource type allocated to each process, and the column request denotes the number of units of each resource type requested by a process in order to complete execution. Which of these processes will finish LAST?

	alloc			request		
	X	Y	Z	X	Y	Z
P0	1	2	1	1	0	3
P1	2	0	1	0	1	2
P2	2	2	1	1	2	0

- (a) P0
- (b) P1
- (c) P2
- (d) None of the above, since the system is in a deadlock

[2007 : 2 M]

3.9 The following is a code with two threads, producer and consumer, that can run in parallel. Further, S and Q are binary semaphores equipped with the standard P and V operations.

semaphore S = 1, Q = 0;

integer x ;

producer :

while (true) do

P(S) ;

x = produce() ;

V(Q) ;

done

consumer :

While (true) do

P(Q) ;

consume (x);

V(S) ;

done

Which of the following is TRUE about the program above?

- (a) The process can deadlock
- (b) One of the threads can starve
- (c) Some of the items produced by the producer may be lost
- (d) Values generated and stored in 'x' by the producer will always be consumed before the producer can generate a new value

[2008 : 2 M]

3.10 An operating system implements a policy that requires a process to release all resources before making a request for another resource.

Select the TRUE statement from the following:

- (a) Both starvation and deadlock can occur
- (b) Starvation can occur but deadlock cannot occur
- (c) Starvation cannot occur but deadlock can occur
- (d) Neither starvation nor deadlock can occur

[2008 : 2 M]

3.11 Which of the following is NOT true of deadlock prevention and deadlock avoidance schemes?

- (a) In deadlock prevention, the request for resources is always granted if the resulting state is safe
- (b) In deadlock avoidance, the request for resources is always granted if the resulting state is safe
- (c) Deadlock avoidance is less restrictive than deadlock prevention
- (d) Deadlock avoidance requires knowledge of resource requirements a priori

[2008 : 2 M]

3.12 Consider a system with 4 types of resources R1 (3 units), R2 (2 units), R3 (3 units), R4 (2 units). A non-preemptive resource allocation policy is used. At any given instance, a request is not entertained if it cannot be completely satisfied. Three processes P1, P2, P3 request the resources as follows if executed independently.

Process P1 :	Process P2 :	Process P3 :
t = 0 : requests 2 units of R2	t = 0 : requests 2 units of R3	t = 0 : requests 1 unit of R4
t = 1 : requests 1 unit of R3	t = 2 : requests 1 unit of R4	t = 2 : requests 2 units of R1
t = 3 : requests 2 units of R1	t = 4 : requests 1 unit of R1	t = 5 : releases 2 units of R1
t = 5 : releases 1 unit of R2 and 1 unit of R1	t = 6 : releases 1 unit of R3	t = 7 : requests 1 unit of R2
t = 7 : releases 1 unit of R3	t = 8 : Finishes	t = 8 : requests 1 unit of R3
t = 8 : requests 2 units of R4		t = 9 : Finishes
t = 10 : Finishes		

Which one of the following statements is TRUE if all three processes run concurrently starting at time $t = 0$?

- (a) All processes will finish without any deadlock
 (b) Only P1 and P2 will be in deadlock
 (c) Only P1 and P3 will be in deadlock
 (d) All three processes will be in deadlock

[2009 : 2 M]

3.13 A system has n resources $R_0, R_1, \dots, R_{n-1}$, and k processes $P_0, P_1, \dots, P_{k-1}$. The implementation of the resource request logic of each process P_i , is as follows:

```

if (i%2 == 0) {
    if (i < n) request  $R_i$ ;
    if (i + 2 < n) request  $R_{i+2}$ ;
}
else {
    if (i < n) request  $R_{n-i}$ ;
    if (i + 2 < n) request  $R_{n-i-2}$ ;
}

```

In which one of the following situations is a deadlock possible?

- (a) $n = 40, k = 26$ (b) $n = 21, k = 12$
 (c) $n = 20, k = 10$ (d) $n = 41, k = 19$

[2010 : 2 M]

3.14 An operating system uses the Banker's algorithm for deadlock avoidance when managing the allocation of three resource types X, Y , and Z to three processes P_0, P_1 , and P_2 . The table given below presents the current system state. Here, the Allocation matrix shows the current number of resources of each type allocated to each process and the Max matrix shows the maximum number of resources of each type required by each process during its execution.

	Allocation			Max		
	X	Y	Z	X	Y	Z
P0	0	0	1	8	4	3
P1	3	2	0	6	2	0
P2	2	1	1	3	3	3

There are 3 units of type X , 2 units of type Y and 2 units of type Z still available. The system is currently in a **safe** state. Consider the following independent requests for additional resources in the current state:

REQ1: P_0 requests 0 units of X , 0 units of Y and 2 units of Z

REQ2: P_1 requests 2 units of X , 0 units of Y and 0 units of Z

Which one of the following is **TRUE**?

- (a) Only REQ1 can be permitted.
 (b) Only REQ2 can be permitted.
 (c) Both REQ1 and REQ2 can be permitted.
 (d) Neither REQ1 nor REQ2 can be permitted.

[2014 (Set-1) : 2 M]

3.15 A system contains three programs and each requires three tape units for its operation. The minimum number of tape units which the system must have such that deadlocks never arise is ____.

[2014 (Set-3) : 2 M]

3.16 A system has 6 identical resources and N processes competing for them. Each process can request at most 2 resources. Which one of the following values of N could lead to a deadlock?

- (a) 1 (b) 2
 (c) 3 (d) 4

[2015 (Set-2) : 1 M]

3.17 Consider the following policies for preventing deadlock in a system with mutually exclusive resources.

- Processes should acquire all their resources at the beginning of execution. If any resource is not available, all resources acquired so far are released.
- The resources are numbered uniquely, and processes are allowed to request for resources only in increasing resource numbers.
- The resources are numbered uniquely, and processes are allowed to request for resources only in decreasing resource numbers.
- The resources are numbered uniquely. A process is allowed to request only for a resource with resource number larger than its currently held resources.

Which of the above policies can be used for preventing deadlock?

- (a) Any one of I and III but not II or IV
 (b) Any one of I, III and IV but not II
 (c) Any one of II and III but not I or IV
 (d) Any one of I, II, III and IV

[2015 (Set-3) : 2 M]

3.18 A system shares 9 tape drives. The current allocation and maximum requirement of tape drives for three processes are shown below:

Process	Current Allocation	Maximum Requirement
P1	3	7
P2	1	6
P3	3	5

Which of the following best describes current state of the system?

- (a) Safe, Deadlocked
 (b) Safe, Not Deadlocked
 (c) Not Safe, Deadlocked
 (d) Not Safe, Not Deadlocked

[2017 (Set-2) : 2 M]

3.19 Consider a system with 3 processes that share 4 instances of the same resource type. Each process can request a maximum of K instances. Resource instances can be requested and released only one at a time. The largest value of K that will always avoid deadlock is _____.

[2018 : 1 M]

3.20 In a system, there are three types of resources: E, F and G. Four processes P_0, P_1, P_2 and P_3 execute concurrently. At the outset, the processes have declared their maximum resource requirements using a matrix named Max as given below. For example $\text{Max}[P_2, F]$ is the maximum number of instances of F that P_2 would require. The number of instances of the resources allocated to the various processes at any given state is given by a matrix named Allocation.

Consider a state of the system with the Allocation matrix as shown below and in which 3 instances of E and 3 instances of F are the only resources available.

Allocation				Max			
	E	F	G		E	F	G
P_0	1	0	1	P_0	4	3	1
P_1	1	1	2	P_1	2	1	4
P_2	1	0	3	P_2	1	3	3
P_3	2	0	0	P_3	5	4	1

From the perspective of deadlock avoidance, which one of the following is true?

- The system is in safe state.
- The system is not in safe state, but would be safe if one more instance of E were available.
- The system is not in safe state, but would be safe if one more instance of F were available.
- The system is not in safe state, but would be safe if one more instance of G were available.

[2018 : 2 M]

3.21 Which of the following statements is/are TRUE with respect to deadlocks?

- Circular wait is a necessary condition for the formation of deadlock.
- In a system where each resource has more than one instance, a cycle in its wait-for graph indicates the presence of a deadlock.
- If the current allocation of resources to processes leads the system to unsafe state, then deadlock will necessarily occur.

- In the resource-allocation graph of a system, if every edge is an assignment edge, then the system is not in deadlock state.

[2022 : 1 M]

3.22 $P = \{P_1, P_2, P_3, P_4\}$ consists of all active processes in an operating system.

$R = \{R_1, R_2, R_3, R_4\}$ consists of single instances of distinct types of resources in the system.

The resource allocation graph has the following assignment and claim edges.

Assignment edges: $R_1 \rightarrow P_1, R_2 \rightarrow P_2, R_3 \rightarrow P_3, R_4 \rightarrow P_4$ (the assignment edge $R_1 \rightarrow P_1$ means resource R_1 is assigned to process P_1 , and so on for others)

Claim edges: $P_1 \rightarrow R_2, P_2 \rightarrow R_3, P_3 \rightarrow R_1, P_2 \rightarrow R_4, P_4 \rightarrow R_2$ (the claim edge $P_1 \rightarrow R_2$ means process P_1 is waiting for resource R_2 , and so on for others)

Which of the following statement(s) is/are CORRECT?

- Aborting P_1 makes the system deadlock free.
- Aborting P_3 makes the system deadlock free.
- Aborting P_2 makes the system deadlock free.
- Aborting P_1 and P_4 makes the system deadlock free.

[2025 (Set-2) : 2 M]

3.23 With respect to deadlocks in an operating system, which of the following statements is/are FALSE?

- Banker's algorithm is used to prevent deadlocks
- Deadlock formation can be prevented by ensuring that the hold and wait condition is not allowed
- An assignment edge in a resource allocation graph is marked from a process to a resource
- A safe state guarantees that all processes can finish without formation of a deadlock

[2026 (Set-1) : 1 M]

3.24 Consider a system consisting of k instances of a resource R , being shared by 5 processes. Assume that each process requires a maximum of two instances of resource R and a process can request or release only one instance at a time. Further, a process can request the second instance of the resource only after acquiring the first instance. The minimum value of K for the system to be deadlock-free is _____. (Answer in integer)

[2026 (Set-1) : 1 M]

Answers Deadlock

3.1 (c)	3.2 (b)	3.3 (c)	3.4 (a)	3.5 (c)	3.6 (a)	3.7 (b)	3.8 (c)	3.9 (d)
3.10 (b)	3.11 (a)	3.12 (a)	3.13 (b)	3.14 (b)	3.15 (7)	3.16 (*)	3.17 (d)	3.18 (b)
3.19 (2)	3.20 (a)	3.21 (a, d)	3.22 (c, d)	3.23 (a, c)	3.24 (6)			

Explanations Deadlock**3.1 (c)**

There are 3 processes, and demands '2' resources. If any one gets equal resources as its demand, deadlock won't occur, and according to this question 4 resources are enough.

3.2 (b)

Every process needs 2 resources. With 5 resources atleast one can get whole 2 and executes. So after execution it frees 2 resources which can be used by other processes.

P_1	P_2	P_3	P_4	P_5
1	1	1	1	1

3.3 (c)

It is not a valid deadlock prevention scheme.

3.4 (a)

Given a deadlock prevention scheme, where each process is assigned a unique time stamp and if killed, it get restarted with same time stamp.

P_h : Process holding resource R.



P_r : Process requesting a resource R.



Now, consider the following scenario, where $P_1, P_2, \dots, P_n$ are n-processes with given time stamp.

Process	P_1	P_2	P_3	P_4	P_5		P_n
Time stamp	10	20	30	40	50		

Case-I: Let P_3 is holding resource R

($\therefore P_h = P_3$) and P_5 is requesting for resource R (so, $P_r = P_5$)

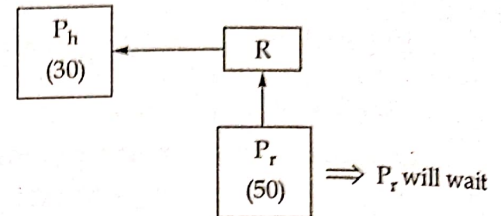
$\therefore$ Condition

if $T(P_r) < T(P_h)$

i.e. $T(P_5) < T(P_3)$

$50 < 30$ // False

Hence P_5 will wait for P_3 .



Case-II: Now consider P_3 is requesting for resource R currently hold by process P_5 , so checking condition.

if $T(P_r) < T(P_h)$

i.e. $T(P_3) < T(P_5)$

$30 < 50$ // True

$\therefore$ Kill P_r

So, P_3 will be killed and P_5 get resource R now suppose after sometime.

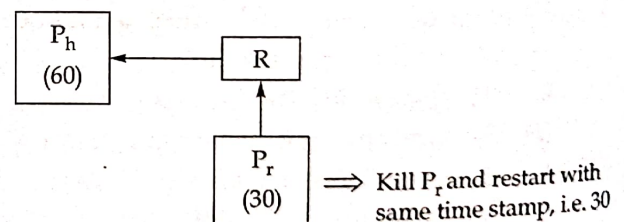
Process P_3 again requesting for resource R let now R is hold by P_6

Again checking

if $T(P_r) < T(P_h)$

$30 < 60$ // True

$\Rightarrow P_3$ will get killed



P_3 will be killed again and similarly if $P_7, P_8, \dots$ so on keep on arriving and holding resource R then innocent process P_3 may go to indefinite waiting which lead to starvation. So, option (a) is answer.

3.5 (c)

Total number of resources shared by process $P_1, \dots, P_n = m$

Maximum resource requirement of process $P_i = S_i$

i.e.

Process	Requirement
P_1	S_1
P_2	S_2
$\vdots$	
P_n	S_n

So the worst case allocation of resources for deadlock prevention. So, that deadlock do not occur is:

[Allot the resources to the processes, one less than the actual requirement (worst case deadlock will occur) + 1 (for no deadlock)]

So, we have

$$(S_1 - 1) + (S_2 - 1) \dots (S_n - 1) + 1 \leq m$$

$$\sum_{i=1}^n (S_i - 1) + 1 \leq m$$

$$\sum_{i=1}^n S_i - \sum_{i=1}^n 1 + 1 \leq m$$

$$\sum_{i=1}^n S_i - n + 1 \leq m$$

$$\sum_{i=1}^n S_i \leq m + n - 1$$

$$\sum_{i=1}^n S_i < (m + n)$$

3.6 (a)

Suppose the slots are full $\rightarrow F = 0$.

Now, if Wait(F) and Wait(S) are interchanges and Wait(S) succeeds, The producer will wait for Wait(F) which is never going to succeed as Consumer would be waiting for Wait(S).

So, deadlock can happen.

If Signal(S) and Signal(F) are interchanged in Consumer, deadlock won't happen.

It will just give priority to a producer compared to the next consumer waiting.

3.7 (b)

Total number of processes = n

Process i is holding x_i instances of a resource R for $1 \leq i \leq n$.

Process i holding resource x_i and additional required y_i . There are two processes p and q such that $y_p = y_q = 0$.

For no dead lock can occur

$$x_p + x_q \geq (\max_{k \neq p, q} Y_k)$$

3.8 (c)

Consider the matrix

	alloc	request
	XYZ	XYZ
P ₀	121	103
P ₁	201	012
P ₂	221	120

So the available matrix is (0, 1, 2) and the requirement of process P₁ is (0, 1, 2) so P₁ can use the available resources after releasing the resources the matrix becomes:

	alloc	request
	XYZ	XYZ
P ₀	121	103
P ₁	000	000
P ₂	221	120

In this time available matrix is (2 1 3) and required matrix of P₀ is (1 0 3) so P₀ uses the resources of the releasing the resource. The available matrix is (3, 3, 4) so P₂ will use the resources, P₂ will finish last.

3.9 (d)

Consumer can consume only once the producer has produced the item, and producer can produce (except the first time) only once the consumer has consumed the item.

3.10 (b)

Releasing all resources before making a request for another resource is a technique of deadlock prevention. So, deadlock cannot occur.

However, we cannot say anything about starvation, it may occur or may not.

3.11 (a)

In deadlock prevention, the request for a resource may not be granted even if the resulting state is safe.

3.12 (a)

t	R1(3)	R2(2)	R3(3)	R1(2)
0	3	0	1	1
1	3	0	0	1
2	1	0	0	0
3	1	0	0	0
4	0	0	0	0
5	1	1	0	0
6	1	1	1	0
7	1	0	2	0
8	2	0	2	1
9	2	1	3	0
10				

3.13 (b)

Logic in question:

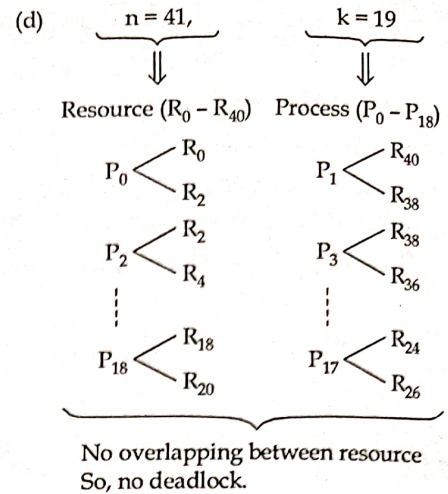
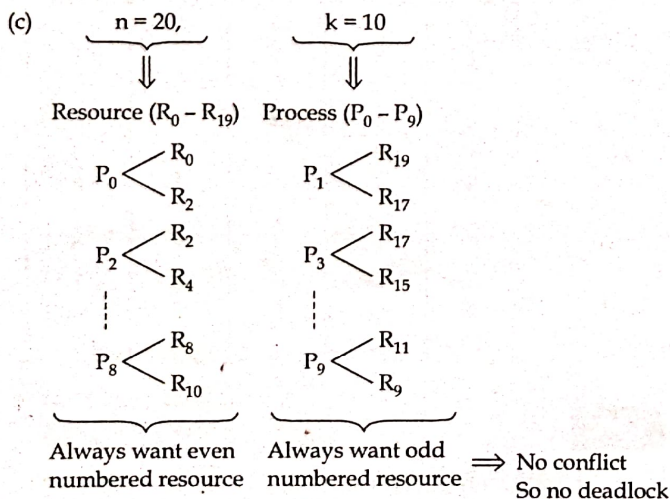
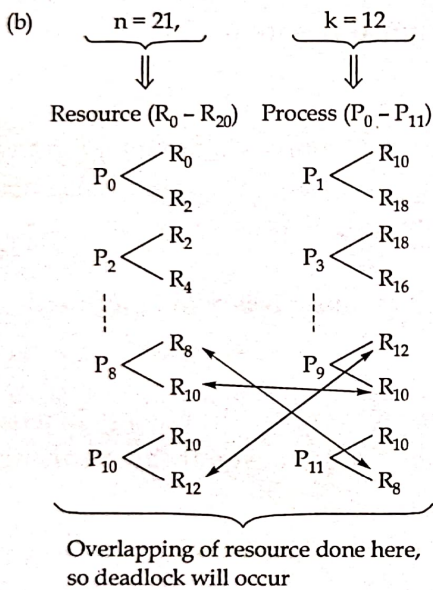
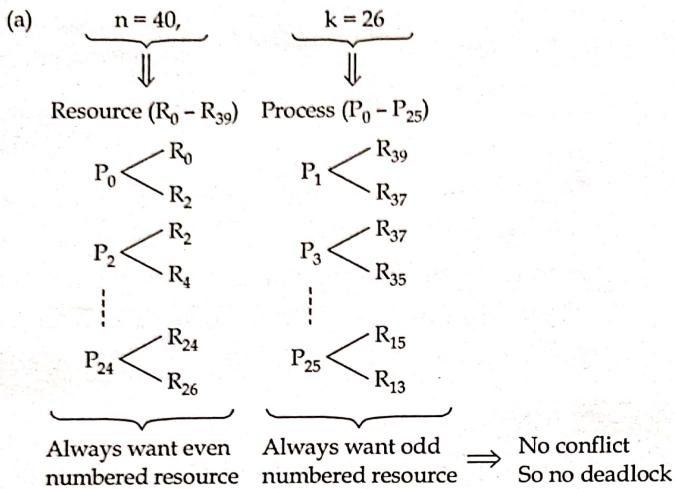
If both odd and even type of process need same resource, then deadlock may occur.

For even processes:

Request : R_i and R_{i+2}

For odd processes:

Request : R_{n-1} and R_{n-i-2}



3.14 (b)

Allocation	Max	Need	Need with additional requests
X Y Z	X Y Z	X Y Z	X Y Z
P0	0 0 1	8 4 3	8 4 2
P1	3 2 0	6 2 0	3 0 0
P2	2 1 1	3 3 3	1 2 2

Available: X Y Z

3 2 2 $\rightarrow$ P2 can satisfy it needs
2 1 1 $\rightarrow$ P2 releases its allocation
5 3 3 $\rightarrow$ P1 satisfies it needs
3 2 0 $\rightarrow$ P1 releases its allocation
8 5 3 $\rightarrow$ P0 can not satisfy with additional requests

$\therefore$ Only REQ2 can be permitted.

3.15 (7)

Number of programs = 3

(P_1, P_2, P_3)

Tape required per program = 3

Allocating tape units, so that deadlock does not occur:

$$\begin{array}{ccc} P_1 & P_2 & P_3 \\ (3-1) & + & (3-1) & + & (3-1) & + & 1 \\ \hline & & & & & & 7 \end{array}$$

Answer is 7.

3.16 (*)

N is number of processes, 6 identical resources

$N \geq 6 \times (2 - 1)$

If $N \geq 6 \Rightarrow$ It could lead to a deadlock

No option matches with $N \geq 6$

- 3.17 (d)
- Dissatisfying hold and wait for deadlock prevention.
 - Dissatisfying circular wait for deadlock prevention.
 - Dissatisfying circular wait for deadlock prevention.
 - Dissatisfying circular wait for deadlock prevention.

3.18 (b)

P.id	Current Allocation	Maximum Requirement	Remaining need	Current available
P1	3	7	4	9 - 7 = 2
P2	1	6	5	5
P3	3	5	2	8
				9

Safe sequence $\Rightarrow P3 \rightarrow P1 \rightarrow P2$

Safe and Not deadlocked

3.19 (2)

P_1	P_2	P_3

No deadlock

Maximum each process can request for 2 resources so, that there will not be any deadlock, because we, have only 4 resource available.

So, K value is '2'.

3.20 (a)

	Max need			Current allocation			Current available			Remaining need		
	E	F	G	E	F	G	E(3)	F(3)	G(0)	E	F	G
P_0	4	3	1	1	0	1	4	3	1	3	3	0
P_1	2	1	4	1	1	2	5	3	4	1	0	2
P_2	1	3	3	1	0	3	6	4	6	0	3	0
P_3	5	4	1	2	0	0	8	4	6	3	4	1

Safe sequence: P_0, P_2, P_1, P_3

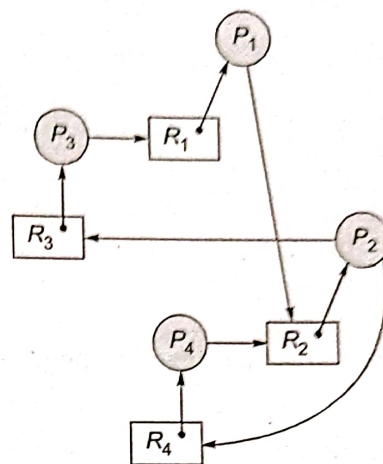
Hence, system is in safe state.

3.21 (a, d)

- Circular wait is one of the necessary condition for the formation of deadlock hence it is true.
- If the resource has more than one instance then cycle is just a necessary condition but not the sufficient condition, hence it is false.
- Unsafe state will not always leads to deadlock, because it depends on behaviour of the processes, hence it is false.

- If every edge is only allocation, it means there is no requirement of the resources, so deadlock not possible, hence it is true.

3.22 (c, d)



Resource Allocation Graph

3.23 (a, c)

- Banker's algorithm ensures that the system never enters an unsafe state by carefully allocating resources. This technique is classified under deadlock avoidance, not prevention. So, option (a) is false.
- Deadlock requires four necessary conditions: mutual exclusion, hold and wait, no preemption, and circular wait. If the hold and wait condition is eliminated, one of the necessary conditions is removed. Hence, deadlock cannot occur. So, option (b) is true.
- In a Resource Allocation Graph (RAG):
 - A request edge goes from process $\rightarrow$ resource.
 - An assignment edge goes from resource $\rightarrow$ process. Thus, the given statement reverses the direction. So, option (c) is false.
- By definition, in a safe state, there exists a safe sequence in which all processes can complete execution. Therefore, deadlock will not occur. So, option (d) is true.

3.24 (6)

$$\begin{array}{cccccc}
 P_1 & P_2 & P_3 & P_4 & P_5 \\
 1 & 1 & 1 & 1 & 1 \\
 1 \\
 = 6
 \end{array}$$

or

$$\begin{aligned}
 m &= n \times (R - 1) + 1 = 5 \times (2 - 1) + 1 \\
 &\Rightarrow 5 + 1 = 6
 \end{aligned}$$

■■■■■